02157 Functional Programming
Week 4:
Algebraic Data Types & Higher-order list functions

Sebastian Mödersheim
Based on slides by Michael Reichhardt Hansen

September 25, 2026

Outline

1. Algebraic Data Types
2. Higher-order list functions

Part I: Disjoint Union – An Example

A *shape* is either a circle, a square, or a triangle

- the union of three disjoint sets

Part I: Disjoint Union – An Example

A *shape* is either a circle, a square, or a triangle

- the union of three disjoint sets

```
type Shape =
  | Circle of float                    (* 1 *)
  | Square of float                    (* 2 *)
  | Triangle of float*float*float;;    (* 3 *)
```

Part I: Disjoint Union – An Example

A *shape* is either a circle, a square, or a triangle

- the union of three disjoint sets

```
type Shape =
  | Circle of float                    (* 1 *)
  | Square of float                    (* 2 *)
  | Triangle of float*float*float;;    (* 3 *)
```

This declaration provides three rules for generating shapes:

- if r : `float`, then `Circle` r : `Shape` (* 1 *)
- if s : `float`, then `Square` s : `Shape` (* 2 *)
- if (a, b, c) : `float*float*float`,
  then `Triangle`(a, b, c) : `Shape` (* 3 *)

Part I: Disjoint Union – An Example

A *shape* is either a circle, a square, or a triangle

- the union of three disjoint sets

```
type Shape =
  | Circle of float                    (* 1 *)
  | Square of float                    (* 2 *)
  | Triangle of float*float*float;;    (* 3 *)
```

This declaration provides three rules for generating shapes:

- if r : `float`, then `Circle` r : `Shape` (* 1 *)
- if s : `float`, then `Square` s : `Shape` (* 2 *)
- if (a, b, c) : `float*float*float`,
  then `Triangle`(a, b, c) : `Shape` (* 3 *)

A type like Shape is also called an algebraic data type

Part I: Disjoint Sets – An Example (II)

The tags `Circle`, `Square` and `Triangle` are called constructors:

$$\begin{array}{lcl} \texttt{Circle} & : & \texttt{float} \rightarrow \texttt{Shape} \\ \texttt{Square} & : & \texttt{float} \rightarrow \texttt{Shape} \\ \texttt{Triangle} & : & \texttt{float} * \texttt{float} * \texttt{float} \rightarrow \texttt{Shape} \end{array}$$

Constructors are functions.

They construct in this case value of `Shape` and they are the only way to construct values of type `Shape`.

```
- Circle 2.0;;
> val it : Shape = Circle 2.0

- Triangle(1.0, 2.0, 3.0);;
> val it : Shape = Triangle(1.0, 2.0, 3.0)

- Square 4.0;;
> val it : Shape = Square 4.0
```

Algebraic Data Types are Inductive Defintions

The declaration

```
type Shape =
  | Circle of float                    (* 1 *)
  | Square of float                    (* 2 *)
  | Triangle of float*float*float;;    (* 3 *)
```

is equivalent to the form of (inductive) definitions we have used for the syntax:

Shape

A Shape is any of the following:

- `Circle(r)` where `r` is a float.
- `Square(a)` where `a` is a float.
- `Triangle(a,b,c)` where `a,b,c` are floats.

and nothing else is a Shape.

This means two things:

- We can define any kind of hierarchical data structures with them (like lists and trees, syntax).
- The constructors can be used for pattern matching because they are the disjoint cases of the data type they construct.

Constructors in Patterns

A Shape can only be constructed by one of the constructors. Thus it always has the form

- `Circle r`
- `Square a`
- `Triangle (a,b,c)`

A shape-area function is declared

```
let area =
  function
  | Circle r         -> System.Math.PI * r * r
  | Square a         -> a * a
  | Triangle(a,b,c) ->
      let s = (a + b + c)/2.0
      sqrt(s*(s-a)*(s-b)*(s-c));;
> val area : Shape -> float
```

following the structure of shapes.

using Heron's formula

Constructors in Patterns

A Shape can only be constructed by one of the constructors. Thus it always has the form

- `Circle r`
- `Square a`
- `Triangle (a,b,c)`

A shape-area function is declared

```
let area =
  function
  | Circle r         -> System.Math.PI * r * r
  | Square a         -> a * a
  | Triangle(a,b,c) ->
      let s = (a + b + c)/2.0
      sqrt(s*(s-a)*(s-b)*(s-c));;
> val area : Shape -> float
```

following the structure of shapes. using Heron's formula

- a constructor only matches itself

Constructors in Patterns

A Shape can only be constructed by one of the constructors. Thus it always has the form

- `Circle r`
- `Square a`
- `Triangle (a,b,c)`

A shape-area function is declared

```
let area =
  function
  | Circle r         -> System.Math.PI * r * r
  | Square a         -> a * a
  | Triangle(a,b,c) ->
      let s = (a + b + c)/2.0
      sqrt(s*(s-a)*(s-b)*(s-c));;
> val area : Shape -> float
```

following the structure of shapes. using Heron's formula

- a constructor only matches itself

$$
\begin{aligned}
& \texttt{area (Circle 1.2)} \\
\rightsquigarrow\ & (\texttt{System.Math.PI * r * r},\ [\texttt{r} \mapsto 1.2]) \\
\rightsquigarrow\ & \ldots
\end{aligned}
$$

Enumeration types – the months

Months are naturally defined using tagged values::

```
type Month = | January | February | March | April
             | May | June | July | August | September
             | October | November | December;;
```

Enumeration types – the months

Months are naturally defined using tagged values::

```
type Month = | January | February | March | April
             | May | June | July | August | September
             | October | November | December;;
```

The days-in-a-month function is declared by

```
let daysOfMonth =
  function
  | February                                -> 28
  | April | June | September | November -> 30
  | _                                       -> 31;;
val daysOfMonth : Month -> int
```

Enumeration types – the months

Months are naturally defined using tagged values::

```
type Month = | January | February | March | April
             | May | June | July | August | September
             | October | November | December;;
```

The days-in-a-month function is declared by

```
let daysOfMonth =
  function
  | February                                -> 28
  | April | June | September | November -> 30
  | _                                       -> 31;;
val daysOfMonth : Month -> int
```

Observe: Constructors need not have arguments

The `option` type

```
type 'a option = None | Some of 'a
```

Distinguishes the cases "nothing" and "something".

predefined

The option type

```
type 'a option = None | Some of 'a
```

Distinguishes the cases "nothing" and "something".

predefined

The constructor Some and None are polymorphic:

```
Some false;;
val it : bool option = Some false

Some (1, "a");;
val it : (int * string) option = Some (1, "a")

None;;
val it : 'a option = None
```

The `option` type

```
type 'a option = None | Some of 'a
```

Distinguishes the cases "nothing" and "something".

predefined

The constructor `Some` and `None` are polymorphic:

```
Some false;;
val it : bool option = Some false

Some (1, "a");;
val it : (int * string) option = Some (1, "a")

None;;
val it : 'a option = None
```

Observe: type variables are allowed in declarations of algebraic types

Example: Find first position of element in a list

`findPos x l`:

- give `None` if `x` does not occur in `l`
- give `Some p` if `p` is the first position where `x` occurs in `l`

Exercise

A (teaching) room at DTU is either an auditorium or a databar:

- an auditorium is characterized by a location and a number of seats.
- a databar is characterized by a location, a number of computers and a number of seats.

Declare a type *Room*.

Exercise

A (teaching) room at DTU is either an auditorium or a databar:

- an auditorium is characterized by a location and a number of seats.
- a databar is characterized by a location, a number of computers and a number of seats.

Declare a type *Room*.

Declare a function:

$$seatCapacity : Room \rightarrow int$$

Exercise

A (teaching) room at DTU is either an auditorium or a databar:

- an auditorium is characterized by a location and a number of seats.
- a databar is characterized by a location, a number of computers and a number of seats.

Declare a type *Room*.

Declare a function:

$$seatCapacity : Room \rightarrow int$$

Declare a function

$$computerCapacity : Room \rightarrow int\ option$$

Lists are an Algebraic Data Type

We can actually define lists using an algebraic data type:

```
type 'a mylist = | Empty
                 | Cons of ('a*'a mylist)
```

To avoid conflicts with the built-in concept of list there are different names:

- `Empty` for `[]`
- `Cons` for `::`
- `mylist` for `list`
- With this data type one would write for instance `Cons (3,Cons(4,Cons(5,Empty)))` for the list `[3;4;5]`.
- We can do all the pattern matching as usual.

Outline

1. Algebraic Data Types
2. **Higher-order list functions**

Motivation

Higher-order functions are

- everywhere

$$\Sigma_{i=a}^{b} f(i),\ \frac{df}{dx},\ \{x \in A \mid P(x)\}, \ldots$$

Motivation

Higher-order functions are

- everywhere

  $\Sigma_{i=a}^{b} f(i),\ \frac{df}{dx},\ \{x \in A \mid P(x)\}, \ldots$

- powerful

  Parameterized modules, succinct code ...

Motivation

Higher-order functions are

- everywhere

  $\Sigma_{i=a}^{b} f(i),\ \frac{df}{dx},\ \{x \in A \mid P(x)\}, \ldots$

- powerful

  Parameterized modules, succinct code ...

HIGHER-ORDER FUNCTIONS ARE USEFUL

Recursion over a List

Many recursive declarations work recursively along a list:

```
let rec f = function
            | []    ->   ...
            | x::xs ->   ... f(xs) ...
```

Many of these functions have a simple common shape:

Applying a function on each list element

```
let rec squarelist = function
  | [] -> []
  | x::xs -> x*x :: squarelist xs
```

Collecting list elements that have a certain property

```
let rec evens = function
  | [] -> []
  | x::xs -> if x%2=0 then x :: evens xs else evens xs
```

Combining all list elements with a function

```
let rec sum = function
  | [] -> 0
  | (x::xs) -> x + sum xs
```

Recursion over a List

Many recursive declarations work recursively along a list:

```
let rec f = function
            | []    ->   ...
            | x::xs ->   ... f(xs) ...
```

Many of these functions have a simple common shape:

Applying a function on each list element

```
let rec squarelist = function
  | [] -> []
  | x::xs -> x*x :: squarelist xs
```

Collecting list elements that have a certain property

```
let rec evens = function
  | [] -> []
  | x::xs -> if x%2=0 then x :: evens xs else evens xs
```

Combining all list elements with a function

```
let rec sum = function
  | [] -> 0
  | (x::xs) -> x + sum xs
```

We can write that more succinct using higher-order functions

Recursion over a List

Many recursive declarations work recursively along a list:

```
let rec f = function
            | []    ->   ...
            | x::xs ->   ... f(xs) ...
```

Applying a function on each list element

```
let squarelist list = List.map (fun x -> x*x) list
```

Collecting list elements that have a certain property

```
let evens list = List.filter (fun x -> x%2=0) list
```

Combining all list elements with a function

```
let sum list = List.fold (+) 0 list
```

This is the more succinct version using the higher-order functions
`List.map` `List.filter` `List.fold`

Recursion over a List

Many recursive declarations work recursively along a list:

```
let rec f = function
            | []    ->   ...
            | x::xs ->   ... f(xs) ...
```

Applying a function on each list element

```
let squarelist list = List.map (fun x -> x*x) list
```

Collecting list elements that have a certain property

```
let evens list = List.filter (fun x -> x%2=0) list
```

Combining all list elements with a function

```
let sum list = List.fold (+) 0 list
```

This is the more succinct version using the higher-order functions
`List.map` `List.filter` `List.fold`

Avoid (almost) identical code fragments by
parameterizing functions with functions

A simple declaration of a list function

A typical declaration following the structure of lists:

```
let rec posList = function
                  | []    -> []
                  | x::xs -> (x > 0)::posList xs;;
val posList : int list -> bool list

posList [4; -5; 6];;
val it : bool list = [true; false; true]
```

A simple declaration of a list function

A typical declaration following the structure of lists:

```
let rec posList = function
                  | []    -> []
                  | x::xs -> (x > 0)::posList xs;;
val posList : int list -> bool list

posList [4; -5; 6];;
val it : bool list = [true; false; true]
```

Applies the function `fun x -> x > 0` to each element in a list

Another declaration with the same structure

```
let rec addElems = function
                   | []         -> []
                   | (x,y)::zs -> (x+y)::addElems zs;;
val addElems : (int * int) list -> int list

addElems [(1,2) ;(3,4)];;
val it : int list = [3; 7]
```

Another declaration with the same structure

```
let rec addElems = function
                   | []         -> []
                   | (x,y)::zs -> (x+y)::addElems zs;;
val addElems : (int * int) list -> int list

addElems [(1,2) ;(3,4)];;
val it : int list = [3; 7]
```

Applies the addition function + to each pair of integers in a list

The function: `map`

Applies a function to each element in a list

$$\texttt{map}\ f\ [v_1; v_2; \ldots; v_n] \;=\; [f(v_1); f(v_2); \ldots; f(v_n)]$$

Declaration

Library function

```
let rec map f = function
                | []    -> []
                | x::xs -> f x :: map f xs;;
val map : ('a -> 'b) -> 'a list -> 'b list
```

The function: `map`

Applies a function to each element in a list

$$\texttt{map}\ f\ [v_1; v_2; \ldots; v_n] \;=\; [f(v_1); f(v_2); \ldots; f(v_n)]$$

Declaration

Library function

```
let rec map f = function
                | []    -> []
                | x::xs -> f x :: map f xs;;
val map : ('a -> 'b) -> 'a list -> 'b list
```

Succinct declarations can be achieved using map, e.g.

```
let posList = map (fun x -> x > 0);;
val posList : int list -> bool list

let addElems = map (fun (x,y) -> x+y);;
val addElems : (int * int) list -> int list
```

Exercise

Declare a function

$$\texttt{mulC}\ c\ [x_1, \ldots, x_n] = [c * x_1, \ldots, c * x_n]$$

Remember

$$\texttt{map}\ f\ [v_1; v_2; \ldots; v_n] \ = \ [f(v_1); f(v_2); \ldots; f(v_n)]$$

where

```
map: ('a -> 'b) -> 'a list -> 'b list
```

Higher-order list functions: `exists`

Predicate: For some x in xs : $p(x)$.

Higher-order list functions: `exists`

Predicate: For some x in xs : $p(x)$.

$$\texttt{exists}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \texttt{true} \text{ for some } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

Higher-order list functions: `exists`

Predicate: For some x in xs : $p(x)$.

$$\texttt{exists}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \texttt{true} \text{ for some } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

Declaration — Library function

```
let rec exists p = function
                   | []    -> false
                   | x::xs -> p x || exists p xs;;
val exists : ('a -> bool) -> 'a list -> bool
```

Higher-order list functions: `exists`

Predicate: For some x in xs : $p(x)$.

$$\texttt{exists}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \texttt{true} \text{ for some } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

Declaration

Library function

```
let rec exists p = function
                   | []    -> false
                   | x::xs -> p x || exists p xs;;
val exists : ('a -> bool) -> 'a list -> bool
```

Example

```
exists (fun x -> x>=2) [1; 3; 1; 4];;
val it : bool = true
```

Exercise

Declare `contains` function using `exists`.

```
let contains x ys = exists ????? ;;
val contains : 'a -> 'a list -> bool when 'a : equality
```

Remember

$$\texttt{exists}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \texttt{true} \text{ for some } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

where

```
exists: ('a -> bool) -> 'a list -> bool
```

contains is a Library function

Higher-order list functions: `forall`

Predicate: For every x in xs : $p(x)$.

Higher-order list functions: `forall`

Predicate: For every x in xs : $p(x)$.

$$\texttt{forall}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \text{true, for all elements } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

Higher-order list functions: `forall`

Predicate: For every x in xs : $p(x)$.

$$\texttt{forall}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \text{true, for all elements } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

Declaration

Library function

```
let rec forall p = function
                   | []    -> true
                   | x::xs -> p x && forall p xs;;
val forall : ('a -> bool) -> 'a list -> bool
```

Higher-order list functions: `forall`

Predicate: For every x in xs : $p(x)$.

$$\texttt{forall}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \text{true, for all elements } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

Declaration

Library function

```
let rec forall p = function
                   | []    -> true
                   | x::xs -> p x && forall p xs;;
val forall : ('a -> bool) -> 'a list -> bool
```

Example

```
forall (fun x -> x>=2) [1; 3; 1; 4];;
val it : bool = false
```

Exercises

Declare a function

`disjoint` *xs ys*

which is true when there are no common elements in the lists *xs* and *ys*, and false otherwise.

Remember

$$\texttt{forall}\ p\ xs = \begin{cases} \text{true} & \text{if } p(x) = \text{true, for all elements } x \text{ in } xs \\ \text{false} & \text{otherwise} \end{cases}$$

where

```
forall : ('a -> bool) -> 'a list -> bool
```

Higher-order list functions: `filter`

Set comprehension: $\{x \in xs : p(x)\}$

Higher-order list functions: `filter`

Set comprehension: $\{x \in xs : p(x)\}$

`filter` p xs is the list of those elements x of xs where $p(x) =$ `true`.

Higher-order list functions: `filter`

Set comprehension: $\{x \in xs : p(x)\}$

`filter` p xs is the list of those elements x of xs where $p(x) =$ `true`.

Declaration — Library function

```
let rec filter p =
  function
  | []    -> []
  | x::xs -> if p x then x :: filter p xs
             else filter p xs;;
val filter : ('a -> bool) -> 'a list -> 'a list
```

Higher-order list functions: `filter`

Set comprehension: $\{x \in xs : p(x)\}$

`filter` p xs is the list of those elements x of xs where $p(x) =$ `true`.

Declaration — Library function

```
let rec filter p =
  function
  | []    -> []
  | x::xs -> if p x then x :: filter p xs
             else filter p xs;;
val filter : ('a -> bool) -> 'a list -> 'a list
```

Example

```
filter System.Char.IsLetter ['1'; 'p'; 'F'; '-'];;
val it : char list = ['p'; 'F']
```

where `System.Char.IsLetter` c is true iff $c \in \{\text{'A'}, \ldots, \text{'Z'}\} \cup \{\text{'a'}, \ldots, \text{'z'}\}$

Exercise

Declare a function

`inter` *xs ys*

which contains the common elements of the lists *xs* and *ys* — i.e. their intersection.

Order and repetition of elements are of no concern

Remember:
`filter` *p xs* is the list of those elements *x* of *xs* where $p(x) = \texttt{true}$. where

```
filter: ('a -> bool) -> 'a list -> 'a list
```

Higher-order list functions: `tryFind`

$$\texttt{tryFind}\ p\ xs = \begin{cases} \texttt{Some}\ x & \text{for an element } x \text{ of } xs \text{ with } p(x) = \text{true} \\ \texttt{None} & \text{if no such element exists} \end{cases}$$

Higher-order list functions: `tryFind`

$$\texttt{tryFind}\ p\ xs = \begin{cases} \texttt{Some}\ x & \text{for an element } x \text{ of } xs \text{ with } p(x) = \text{true} \\ \texttt{None} & \text{if no such element exists} \end{cases}$$

```
let rec tryFind p =
  function
  | x::xs when p x -> Some x
  | _::xs          -> tryFind p xs
  | _              -> None;;
val tryFind : ('a -> bool) -> 'a list -> 'a option
```

Higher-order list functions: `tryFind`

$$\texttt{tryFind}\ p\ xs = \begin{cases} \texttt{Some}\ x & \text{for an element } x \text{ of } xs \text{ with } p(x) = \text{true} \\ \texttt{None} & \text{if no such element exists} \end{cases}$$

```
let rec tryFind p =
  function
  | x::xs when p x -> Some x
  | _::xs          -> tryFind p xs
  | _              -> None;;
val tryFind : ('a -> bool) -> 'a list -> 'a option
```

```
tryFind (fun x -> x>3) [1;5;-2;8];;
val it : int option = Some 5
```

Folding a function over a list (I)

Example: sum of absolute values:

```
let rec absSum = function
                 | []    -> 0
                 | x::xs -> abs x + absSum xs;;
val absSum : int list -> int

absSum [-2; 2; -1; 1; 0];;
val it : int = 6
```

Folding a function over a list (II)

```
let rec absSum = function
               | []    -> 0
               | x::xs -> abs x + absSum xs;;
```

Let $f\ x\ a$ abbreviate $\texttt{abs}\ x + a$ in the evaluation:

$$
\begin{array}{rl}
 & \texttt{absSum}\ [x_0; x_1; \ldots; x_{n-1}] \\
\rightsquigarrow & \texttt{abs}\ x_0 + (\texttt{absSum}\ [x_1; \ldots; x_{n-1}]) \\
= & f\ x_0\ (\texttt{absSum}\ [x_1; \ldots; x_{n-1}]) \\
\rightsquigarrow & f\ x_0\ (f\ x_1\ (\texttt{absSum}[x_2; \ldots; x_{n-1}])) \\
 & \vdots \\
\rightsquigarrow & f\ x_0\ (f\ x_1\ (\cdots (f\ x_{n-1}\ 0) \cdots))
\end{array}
$$

Folding a function over a list (II)

```
let rec absSum = function
                 | []    -> 0
                 | x::xs -> abs x + absSum xs;;
```

Let $f\ x\ a$ abbreviate $\texttt{abs}\ x + a$ in the evaluation:

$$
\begin{array}{rl}
 & \texttt{absSum}\ [x_0; x_1; \ldots; x_{n-1}] \\
\rightsquigarrow & \texttt{abs}\ x_0 + (\texttt{absSum}\ [x_1; \ldots; x_{n-1}]) \\
= & f\ x_0\ (\texttt{absSum}\ [x_1; \ldots; x_{n-1}]) \\
\rightsquigarrow & f\ x_0\ (f\ x_1\ (\texttt{absSum}[x_2; \ldots; x_{n-1}])) \\
 & \vdots \\
\rightsquigarrow & f\ x_0\ (f\ x_1\ (\cdots (f\ x_{n-1}\ 0) \cdots))
\end{array}
$$

This repeated application of f is also called a folding of f.

Folding a function over a list (II)

```
let rec absSum = function
                 | []    -> 0
                 | x::xs -> abs x + absSum xs;;
```

Let $f\ x\ a$ abbreviate $\texttt{abs}\ x + a$ in the evaluation:

$$
\begin{array}{rl}
 & \texttt{absSum}\ [x_0; x_1; \ldots; x_{n-1}] \\
\rightsquigarrow & \texttt{abs}\ x_0 + (\texttt{absSum}\ [x_1; \ldots; x_{n-1}]) \\
= & f\ x_0\ (\texttt{absSum}\ [x_1; \ldots; x_{n-1}]) \\
\rightsquigarrow & f\ x_0\ (f\ x_1\ (\texttt{absSum}[x_2; \ldots; x_{n-1}])) \\
 & \vdots \\
\rightsquigarrow & f\ x_0\ (f\ x_1\ (\cdots (f\ x_{n-1}\ 0) \cdots))
\end{array}
$$

This repeated application of f is also called a folding of f.

Many functions follow such recursion and evaluation schemes

Higher-order list functions: `foldBack` (1)

Suppose that $\otimes$ is an infix function. Then

$$\texttt{foldBack}\ (\otimes)\ [a_0\texttt{;}\ a_1\texttt{;}\ \ldots\ \texttt{;}\ a_{n-2}\texttt{;}\ a_{n-1}]\ e_b = a_0 \otimes (a_1 \otimes (\ldots (a_{n-2} \otimes (a_{n-1} \otimes e_b)) \ldots))$$

Higher-order list functions: `foldBack` (1)

Suppose that $\otimes$ is an infix function. Then

$$\texttt{foldBack}\ (\otimes)\ [a_0;\ a_1;\ \ldots\ ;\ a_{n-2};\ a_{n-1}]\ e_b = a_0 \otimes (a_1 \otimes (\ldots (a_{n-2} \otimes (a_{n-1} \otimes e_b))\ldots))$$

$$\texttt{List.foldBack}\ (+)\ [1;\ 2;\ 3]\ 0 = 1 + (2 + (3 + 0)) = 6$$
$$\texttt{List.foldBack}\ (-)\ [1;\ 2;\ 3]\ 0 = 1 - (2 - (3 - 0)) = 2$$

Declaration of `foldBack`

```
let rec foldBack f xlst e =
  match xlst with
  | x::xs -> f x (foldBack f xs e)
  | []    -> e;;
val foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
```

Declaration of `foldBack`

```
let rec foldBack f xlst e =
  match xlst with
  | x::xs -> f x (foldBack f xs e)
  | []    -> e;;
val foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
```

```
let absSum xs = foldBack (fun x a -> abs x + a)  xs 0;;

let length xs = foldBack (fun _ n -> n+1) xs 0;;

let map f xs = foldBack (fun x rs -> f x :: rs) xs [];;
```

Exercise

Let an insertion function be declared by

```
let insert x ys = if List.contains x ys then ys
                  else x::ys;;
```

Declare a function `distinct` xs, that returns a list having same elements as xs but without duplicated element.

Remember:

$$\texttt{foldBack}\ (\oplus)\ [x_1;x_2;\ldots;x_n]\ b \ \rightsquigarrow\ x_1 \oplus (x_2 \oplus \cdots \oplus (x_n \oplus b)\cdots)$$

where

```
foldBack: ('a -> 'state -> 'state)
             -> 'a list -> 'state -> 'state
```

Higher-order list functions: `fold` (1)

Suppose that $\oplus$ is an infix function.

Then the `fold` function is defined by:

$$\texttt{fold}\ (\oplus)\ e_a\ [b_0;\ b_1;\ \ldots\ ;\ b_{n-2};\ b_{n-1}\ ] \\ = \quad ((\ldots((e_a \oplus b_0) \oplus b_1)\ldots) \oplus b_{n-2}) \oplus b_{n-1}$$

i.e. it applies $\oplus$ from left to right.

Higher-order list functions: `fold` (1)

Suppose that $\oplus$ is an infix function.

Then the `fold` function is defined by:

$$\texttt{fold}\ (\oplus)\ e_a\ [b_0;\ b_1;\ \ldots\ ;\ b_{n-2};\ b_{n-1}\ ] \\ = \quad ((\ldots((e_a \oplus b_0) \oplus b_1)\ldots) \oplus b_{n-2}) \oplus b_{n-1}$$

i.e. it applies $\oplus$ from left to right.

Examples:

$$\texttt{List.fold (-) 0 [1; 2; 3]} \quad = \quad ((0-1)-2)-3 \quad = \quad -6$$

Higher-order list functions: `fold` (1)

Suppose that $\oplus$ is an infix function.

Then the `fold` function is defined by:

$$\texttt{fold } (\oplus)\ e_a\ [b_0;\ b_1;\ \ldots\ ;\ b_{n-2};\ b_{n-1}\ ] \\ = \quad ((\ldots((e_a \oplus b_0) \oplus b_1)\ldots) \oplus b_{n-2}) \oplus b_{n-1}$$

i.e. it applies $\oplus$ from left to right.

Examples:

$$\texttt{List.fold (-) 0 [1; 2; 3]} = ((0-1)-2)-3 = -6$$
$$\texttt{List.foldBack (-) [1; 2; 3] 0} = 1-(2-(3-0)) = 2$$

Higher-order list functions: `fold` (2)

```
let rec fold f e =
  function
  | x::xs -> fold f (f e x) xs
  | []    -> e;;
val fold : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
```

Higher-order list functions: `fold` (2)

```
let rec fold f e =
  function
  | x::xs -> fold f (f e x) xs
  | []    -> e;;
val fold : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
```

Using cons in connection with fold gives the reverse function:

```
let rev xs  = fold (fun rs x -> x::rs) [] xs;;
```

This function has a linear execution time:

Higher-order list functions: `fold` (2)

```
let rec fold f e =
  function
  | x::xs -> fold f (f e x) xs
  | []    -> e;;
val fold : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
```

Using cons in connection with fold gives the reverse function:

```
let rev xs  = fold (fun rs x -> x::rs) [] xs;;
```

This function has a linear execution time:

```
    rev [1;2;3]
⇝   fold   (fun ...)  []          [1;2;3]
⇝   fold   (fun ...)  (1::[])     [2;3]
⇝   fold   (fun ...)  [1]         [2;3]
⇝   fold   (fun ...)  (2::[1])    [3]
⇝   fold   (fun ...)  [2;1]       [3]
⇝   fold   (fun ...)  (3::[2;1])  []
⇝   fold   (fun ...)  [3;2;1]     []
⇝   [3;2;1]
```

Summary

Part I: Disjoint union (Algebraic data types)
– types containing different kinds of values

Summary

Part I: Disjoint union (Algebraic data types)

– types containing different kinds of values

We will later extend the notion with recursive definitions

– provides a mean to declare types for finite trees

Summary

Part I: Disjoint union (Algebraic data types)
– types containing different kinds of values

We will later extend the notion with recursive definitions
– provides a mean to declare types for finite trees

Part II: Higher-order list functions

- Many recursive declarations follows the same schema.

Succinct declarations achievable using higher-order functions

Summary

Part I: Disjoint union (Algebraic data types)
– types containing different kinds of values

We will later extend the notion with recursive definitions
– provides a mean to declare types for finite trees

Part II: Higher-order list functions

- Many recursive declarations follows the same schema.

  Succinct declarations achievable using higher-order functions

- Higher-order list functions (in the library)
  - ★ map
  - ★ contains, exists, forall, filter, tryFind
  - ★ foldBack, fold

Summary

Part I: Disjoint union (Algebraic data types)
– types containing different kinds of values

We will later extend the notion with recursive definitions
– provides a mean to declare types for finite trees

Part II: Higher-order list functions

- Many recursive declarations follows the same schema.

  Succinct declarations achievable using higher-order functions

- Higher-order list functions (in the library)
  - ★ map
  - ★ contains, exists, forall, filter, tryFind
  - ★ foldBack, fold

Avoid (almost) identical code fragments by
parameterizing functions with functions

Exercises

- HR 4.18, on paper - again understanding the type of a function, this time higher order.
- Task 2 from Summer exam 2015
  https://www.imm.dtu.dk/ mire/FSharpBook/oldExams.html
- As a larger program: an airport luggage system based on an old exam question. See next slides.

Exercises

Airport Luggage system

- Every piece of luggage is marked with an *identification*:

  ```
  type Lid = string
  ```

- Each piece of lugagge will have a *route*, i.e. a list of *flights* and *destination* airports:

  ```
  type Flight = string
  type Airport = string
  type Route = (Flight * Airport) list
  ```

- A *luggage catalogue* of an airport associates with each piece of lugagge its route as a pair of an `Lid` and a `Route`:

  ```
  type LuggageCatalogue = (Lid * Route) list
  ```

An example of a luggage catalogue is

```
[("DL 016-914", [("DL 189","ATL"); ("DL 124","BRU"); ("SN 733","CPH")]);
("SK 222-142", [("SK 208","ATL"); ("DL 124","BRU"); ("SK 122","JFK")])]
```

where first element in the list describes that the piece of luggage with identification `DL 016-914` is following a route, where it is first flown to Atlanta (`ATL`) with flight `DL 189`, then flown to Bruxelles (`BRU`) with flight `DL 124`, and so on.

Exercises

Airport Luggage system

For all the following functions you can give recursive definitions, but in each case there is also an easy way to use instead the built-in higher-order functions on lists like `List.fold` we discussed in the lecture.

1. Declare a function `findRoute: Lid -> LuggageCatalogue -> Route list`, that finds all route(s) for a given luggage identification in a luggage catalogue.

   We use here a list of routes as result because the given `Lid` may not be found. Also, if the luggage catalogue accidentally has several entries for the same `Lid`, you may return all routes or just the first or stop with an error.

2. Declare a function `inRoute: Flight -> Route -> bool` that decides whether a given flight occurs in a route.

3. Declare a function `withFlight f lc`, where `f` is a flight and `lc` is a luggage catalogue. It returns a list of luggage identifiers for the pieces of luggage that should travel with `f` according to `lc`.
   For the example catalogue, `DL 124` gives both `DL 016-914` and `SK 222-142`.

Exercises

Airport Luggage system

An arrival catalogue associates with every airport the identifications of all pieces of luggage that should arrive at the airport:

```
type ArrivalCatalogue = (Airport * Lid list) list
```

For the example luggage catalogue, the arrival catalogue is:

```
[("ATL", ["DL 016-914"; "SK 222-142"]); ("BRU", ["DL 016-914"; "SK 222-142"]);
 ("JFK", ["SK 222-142"]); ("CPH", ["DL 016-914"])]
```

To compute it, declare the following functions – also these can be programmed succinctly using higher-order list functions:

- `insertLA lid ap ac` takes an arrival catalogue `ac` and returns a new arrival catalogue that is augmented by the fact that luggage `lid` will arrive at airport `ap`. Note that you have to handle both the case that airport `ap` does not yet occur in the given `ac`, and the case that it does.
- `extend: Lid*Route -> ArrivalCatalogue -> ArrivalCalalogue` so that `extend (lid, r) ac` is the arrival catalogue obtained by extending `ac` with the information that `lid` will arrive at each airport contained in route `r`. This function can make good use of the function `insertLA`.
- Declare a function `toArrivalCatalogue: LuggageCatalogue -> ArrivalCatalogue`, that creates an arrival catalogue from the information of a given luggage catalogue. This function can make good use the function `extend`.